1. Conditional Expressions Evaluation

Conditional expressions in Python use **if-else** statements to evaluate conditions.

List comprehension offers a shorter syntax when you want to create a new list based on the values of an existing list.

Syntax of a Conditional Expression

```
result = value_if_true if condition else value_if_false
```

Example

```
x = 10
y = 20
min_value = x if x < y else y
print(min_value)  # Output: 10
```

- The expression **`x if x < y else y`** evaluates to `x` because `x < y` is `True`.

Multiple Conditions in Conditional Expression

```
status = "Positive" if x > 0 else "Zero" if x == 0
else "Negative"
```

This is equivalent to:

```
if x > 0:
    status = "Positive"
elif x == 0:
    status = "Zero"
else:
    status = "Negative"
```

---

2. Float Representation

- Python represents floating-point numbers using **IEEE 754 (double-precision 64-bit format)**.
- Floats may not be exact due to binary approximation.

Example of Floating-Point Precision Issue

```
print(0.1 + 0.2)  # Output: 0.30000000000000004
```

- This happens because **0.1 and 0.2** cannot be exactly represented in binary.

Solving Precision Issues

Using the `decimal` module:

```
from decimal import Decimal

print(Decimal('0.1') + Decimal('0.2'))  # Output: 0.3
```

Using `round()`:

```
print(round(0.1 + 0.2, 2))  # Output: 0.3
```

---

3. Boolean Type

- Boolean type has **two values**: `True` and `False`.
- Internally, `True` is represented as `1`, and `False` as `0`.

Boolean Conversion

```
print(bool(0))  # False
print(bool(1))  # True
print(bool(""))  # False (empty string is considered False)
print(bool("Hello"))  # True (non-empty string is True)
print(bool([]))  # False (empty list)
print(bool([1, 2, 3]))  # True (non-empty list)
```

---

4. Boolean Operators

Boolean operators work with logical values and return `True` or `False`.

Logical Operators

Operator	Name	Example	Output
`and`	Logical AND	`True and False`	`False`

Operator	Name	Example	Output
`or`	Logical OR	`True or False`	`True`
`not`	Logical NOT	`not True`	`False`

Example

```
a = True
b = False
print(a and b)  # False
print(a or b)   # True
print(not a)    # False
```

Short-Circuit Evaluation

- `and` → If the first operand is `False`, it **stops** and returns `False`.
- `or` → If the first operand is `True`, it **stops** and returns `True`.

```
print(0 and 10)  # Output: 0 (Stops at 0)
print(10 or 0)   # Output: 10 (Stops at 10)
```

---

5. String Operators

Python provides operators for string manipulation.

Concatenation (`+`)

Joins two strings.

```
str1 = "Hello"
str2 = " World"
result = str1 + str2
print(result)  # Output: Hello World
```

Repetition (`*`)

Repeats a string multiple times.

```
print("Python " * 3)  # Output: Python Python Python
```

Membership (`in`, `not in`)

Checks if a substring exists.

```
print("P" in "Python")  # True
print("Java" not in "Python")  # True
```

Indexing

Access characters using index.

```
text = "Python"
print(text[0])  # Output: P
print(text[-1]) # Output: n
```

Slicing

Extracts substring.

```
text = "Programming"
print(text[0:4])  # Output: Prog
print(text[:5])   # Output: Progr
print(text[4:])   # Output: ramming
```

String Comparison (==, !=, <, >)

```
print("apple" < "banana")  # True (Lexicographically)
print("hello" == "hello")  # True
```